BDI-II-U1-VIEWS-1093855059

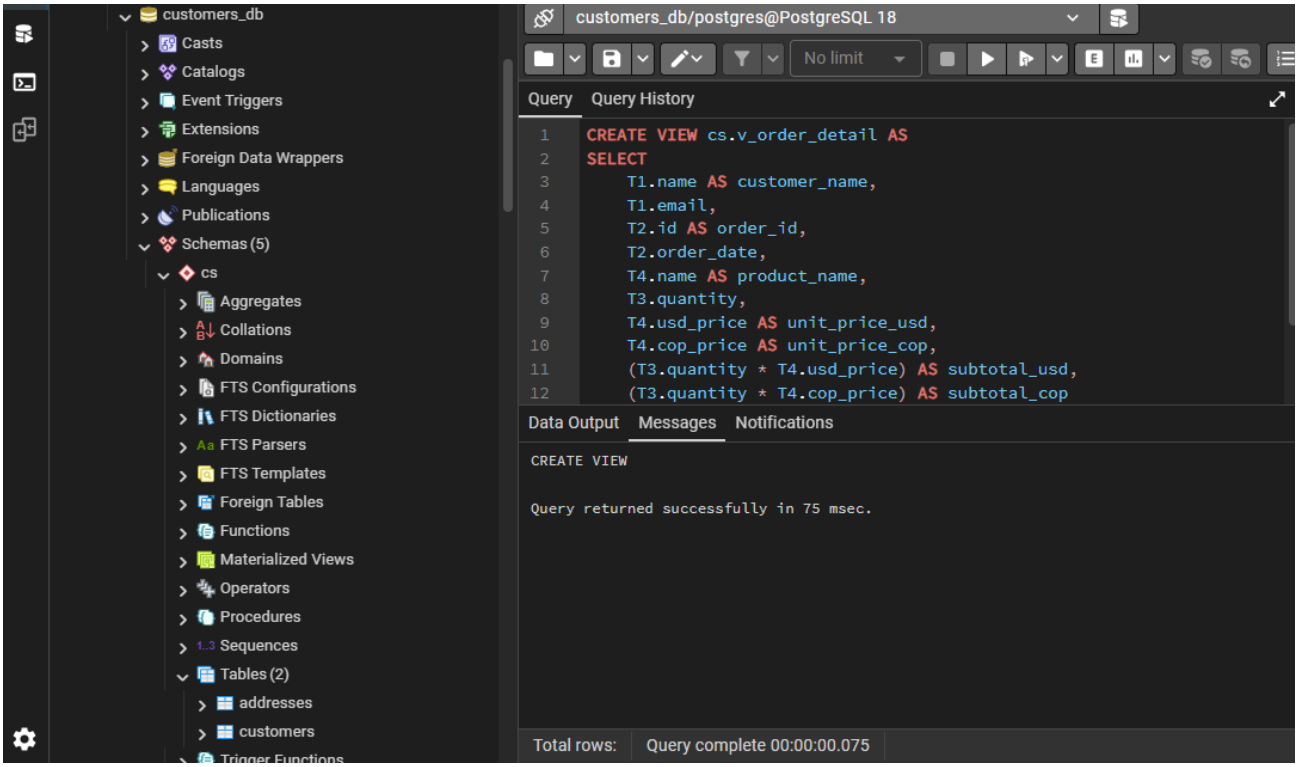
Este documento contiene la resolución del taller de Vistas para la asignatura Bases de Datos II.

Ejercicio 1. Vista normal — Detalle completo de órdenes

a) Crear la vista `cs.v_order_detail`

```
CREATE VIEW cs.v_order_detail AS
SELECT
    T1.name AS customer_name,
    T1.email,
    T2.id AS order_id,
    T2.order_date,
    T4.name AS product_name,
    T3.quantity,
    T4.usd_price AS unit_price_usd,
    T4.cop_price AS unit_price_cop,
    (T3.quantity * T4.usd_price) AS subtotal_usd,
    (T3.quantity * T4.cop_price) AS subtotal_cop
FROM cs.customers T1
INNER JOIN pay.orders T2 ON T1.id_number = T2.customer_id_number
INNER JOIN pay.order_items T3 ON T2.id = T3.order_id
INNER JOIN ctg.products T4 ON T3.product_id = T4.id;
```

Image 1

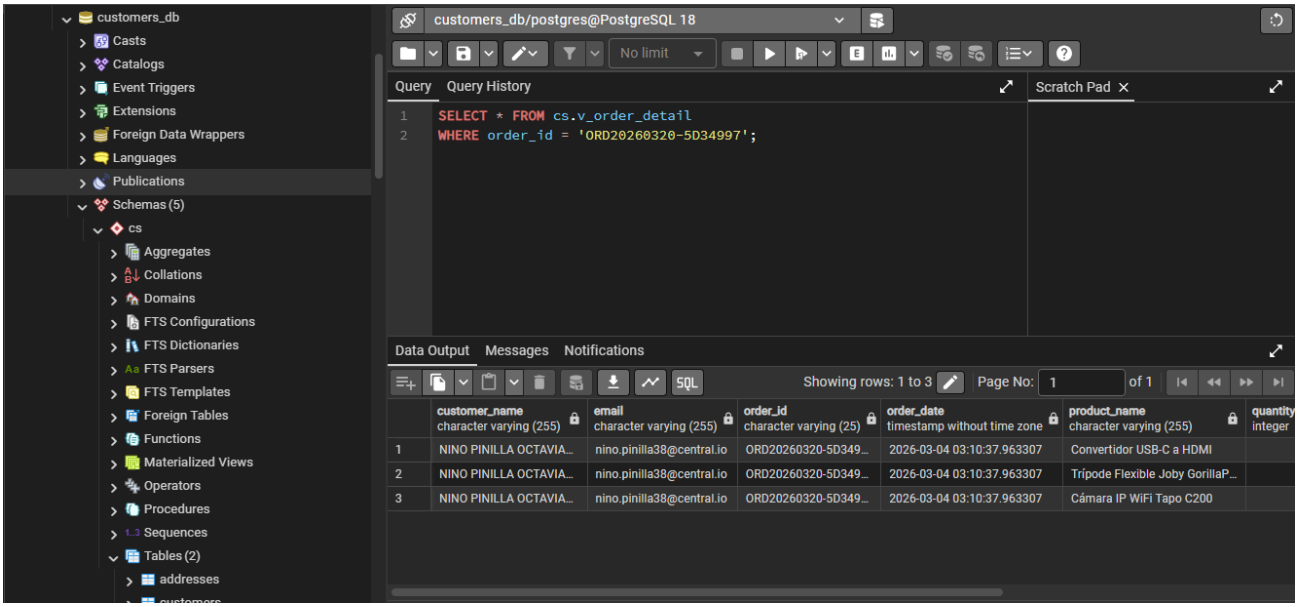


b) Consultar la vista filtrando por un `order_id` **específico**

```
SELECT * FROM cs.v_order_detail
```

```
WHERE order_id = 'ORD20260320-5D34997';
```

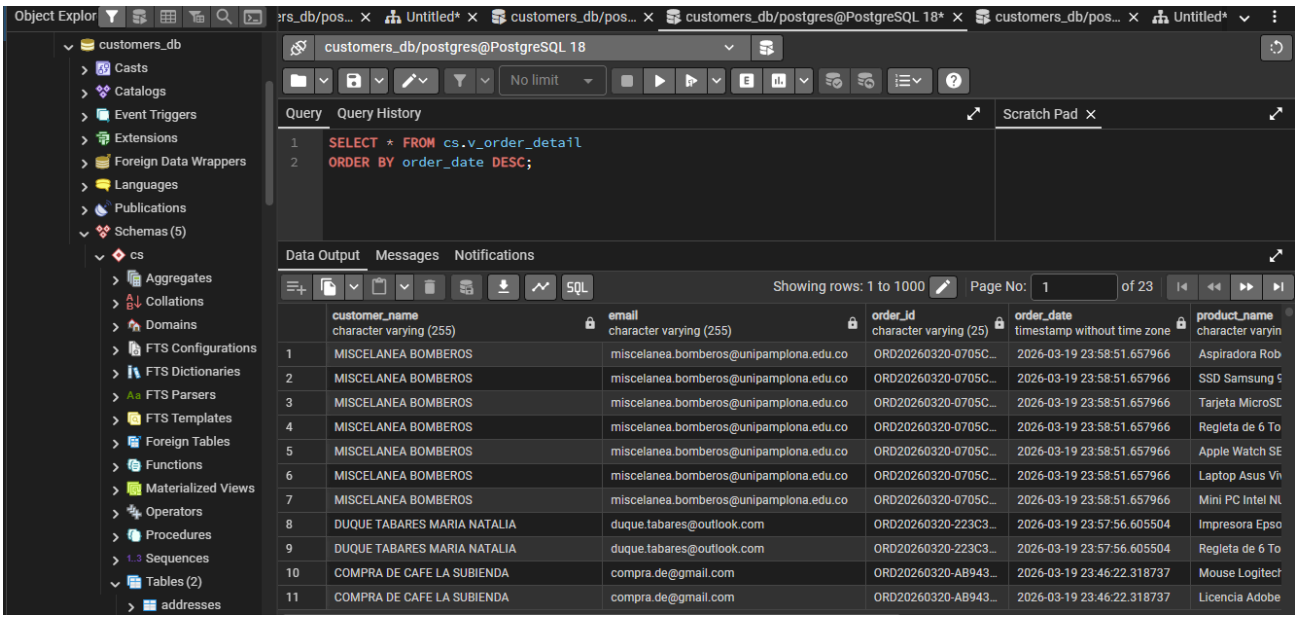
Image 2



c) Consultar la vista ordenando por fecha de orden de manera descendente

```
SELECT * FROM cs.v_order_detail
ORDER BY order_date DESC;
```

Image 3



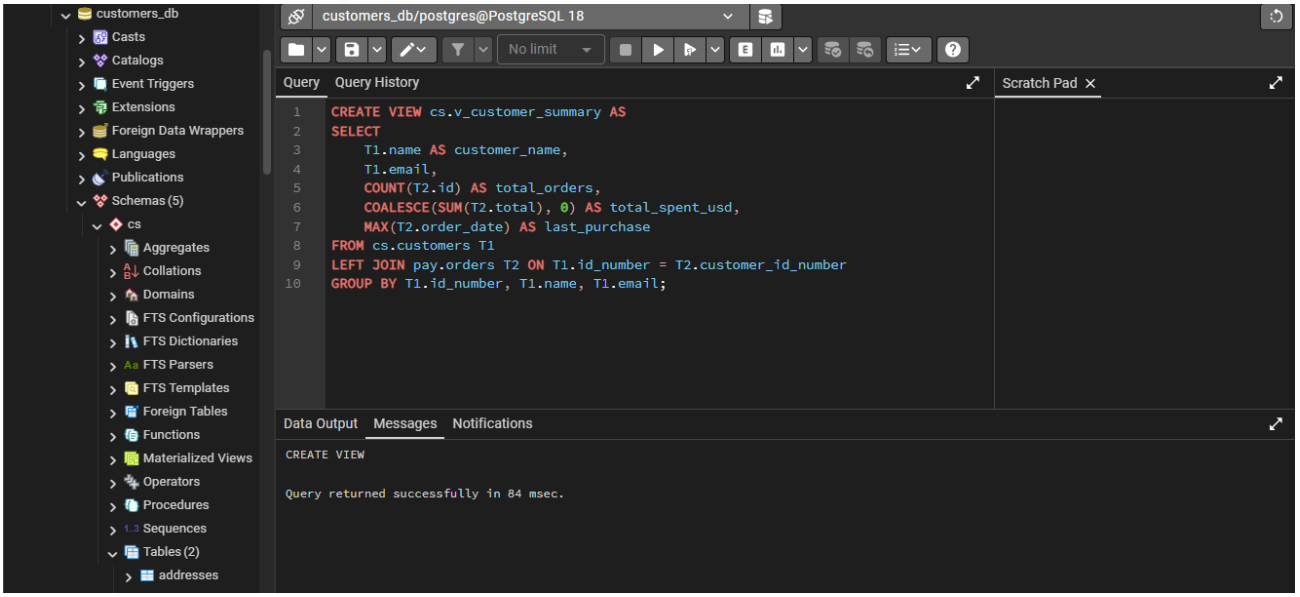
Ejercicio 2. Vista normal — Resumen por cliente

a) Crear la vista `cs.v_customer_summary`

```
CREATE VIEW cs.v_customer_summary AS
SELECT
```

```sql
    T1.name AS customer_name,
    T1.email,
    COUNT(T2.id) AS total_orders,
    COALESCE(SUM(T2.total), 0) AS total_spent_usd,
    MAX(T2.order_date) AS last_purchase
FROM cs.customers T1
LEFT JOIN pay.orders T2 ON T1.id_number = T2.customer_id_number
GROUP BY T1.id_number, T1.name, T1.email;
```

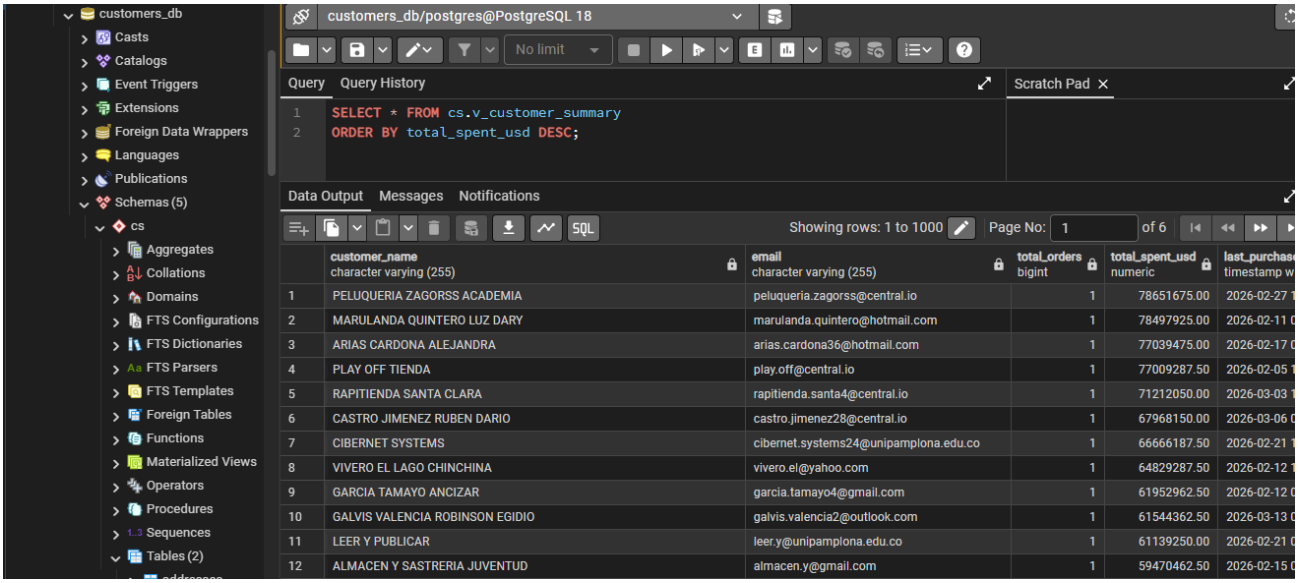
Image 4



b) Consultar la vista ordenando por total gastado de mayor a menor

```sql
SELECT * FROM cs.v_customer_summary
ORDER BY total_spent_usd DESC;
```

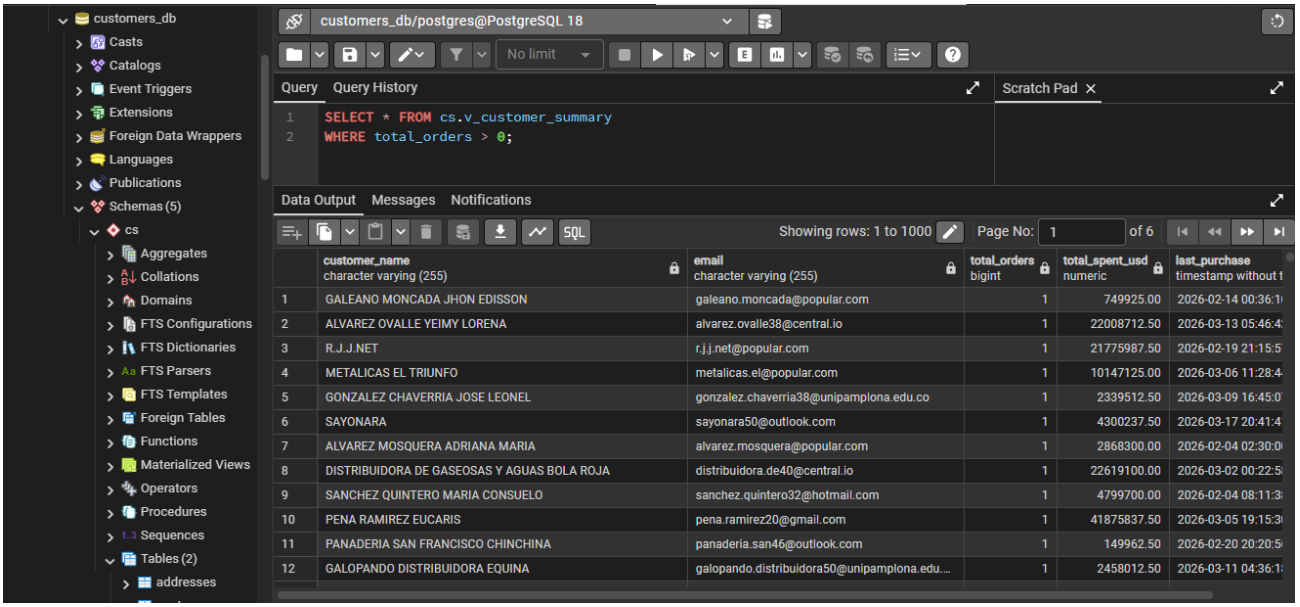
Image 5



c) Consultar únicamente los clientes que tienen al menos una orden registrada

```
SELECT * FROM cs.v_customer_summary
WHERE total_orders > 0;
```

Image 6



Ejercicio 3. Vista normal — Productos con su categoría

a) Crear la vista `cs.v_products_with_category`

```
CREATE VIEW cs.v_products_with_category AS
SELECT
    T1.id AS product_id,
    T1.name AS product_name,
    T1.usd_price,
    T1.cop_price,
    T2.name AS category_name
FROM ctg.products T1
LEFT JOIN ctg.categories T2 ON T1.category_id = T2.id;
```

Image 7

customers_db/postgres@PostgreSQL 18

```
CREATE VIEW cs.v_products_with_category AS
SELECT
    T1.id AS product_id,
    T1.name AS product_name,
    T1.usd_price,
    T1.cop_price,
    T2.name AS category_name
FROM ctg.products T1
LEFT JOIN ctg.categories T2 ON T1.category_id = T2.id;
```

```
CREATE VIEW

Query returned successfully in 110 msec.
```

Total rows: Query complete 00:00:00.110 CRLF Ln 1, Col 43

b) Consultar la vista filtrando únicamente los productos que sí tienen categoría asignada

```
SELECT * FROM cs.v_products_with_category
WHERE category_name IS NOT NULL;
```

Image 8

customers_db/postgres@PostgreSQL 18

```
SELECT * FROM cs.v_products_with_category
WHERE category_name IS NOT NULL;
```

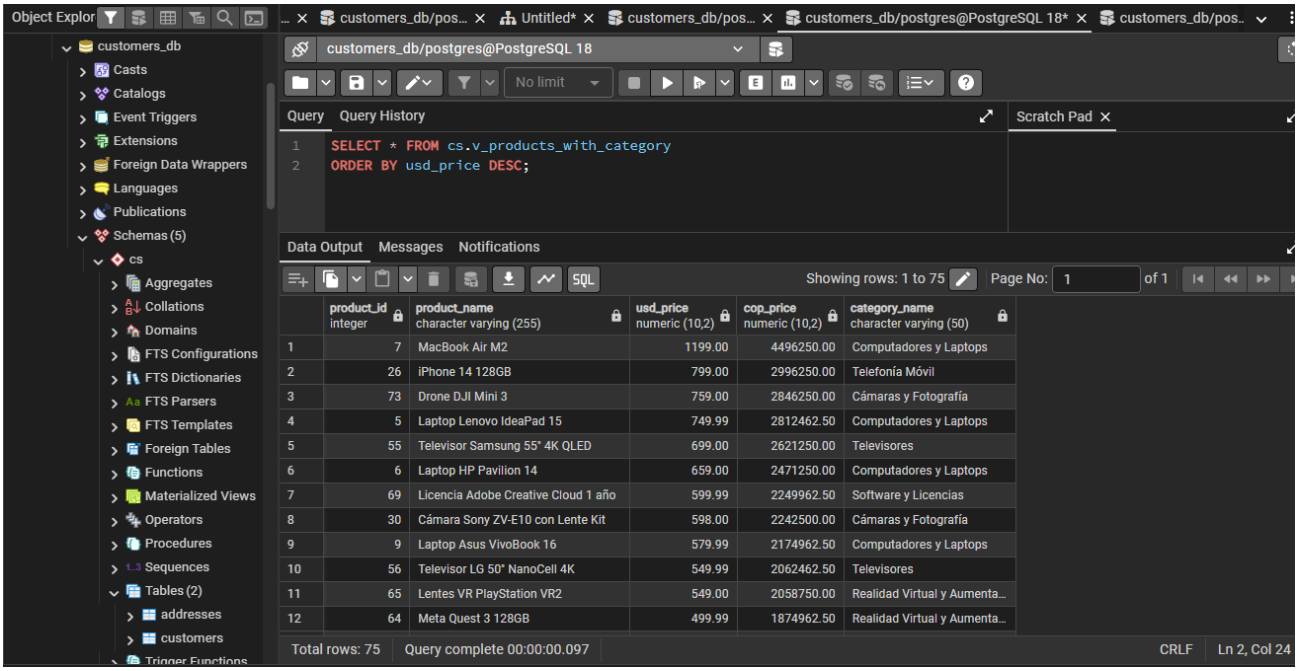
Showing rows: 1 to 75 Page No: 1 of 1

	product_id integer	product_name character varying (255)	usd_price numeric (10,2)	cop_price numeric (10,2)	category_name character varying (50)
1	1	Adaptador de Corriente Universal	25.99	97462.50	Electrónica
2	2	Regleta de 6 Tomas con Protección	18.50	69375.00	Electrónica
3	3	Cable HDMI 2.1 2m	12.99	48712.50	Electrónica
4	4	Convertidor USB-C a HDMI	22.00	82500.00	Electrónica
5	5	Laptop Lenovo IdeaPad 15	749.99	2812462.50	Computadores y Laptops
6	6	Laptop HP Pavilion 14	659.00	2471250.00	Computadores y Laptops
7	7	MacBook Air M2	1199.00	4496250.00	Computadores y Laptops
8	8	Mini PC Intel NUC i5	420.00	1575000.00	Computadores y Laptops
9	9	Laptop Asus VivoBook 16	579.99	2174962.50	Computadores y Laptops
10	10	Mouse Logitech MX Master 3	89.99	337462.50	Periféricos
11	11	Teclado Mecánico Keychron K2	129.00	483750.00	Periféricos
12	12	Teclado Inalámbrico Logitech K380	49.99	187462.50	Periféricos

c) Consultar la vista ordenando por precio en USD de mayor a menor

```
SELECT * FROM cs.v_products_with_category
ORDER BY usd_price DESC;
```

Image 9

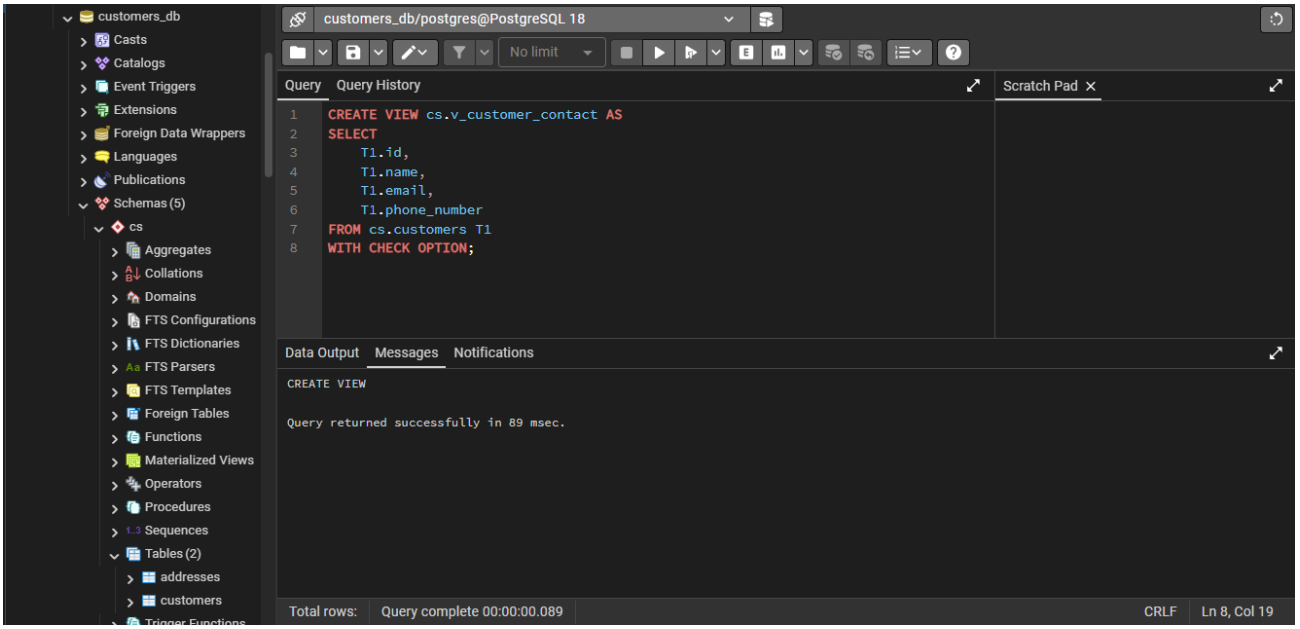


Ejercicio 4. Vista actualizable — Datos de contacto del cliente

a) Crear la vista utilizando la cláusula `WITH CHECK OPTION`

```
CREATE VIEW cs.v_customer_contact AS
SELECT
    T1.id,
    T1.name,
    T1.email,
    T1.phone_number
FROM cs.customers T1
WITH CHECK OPTION;
```

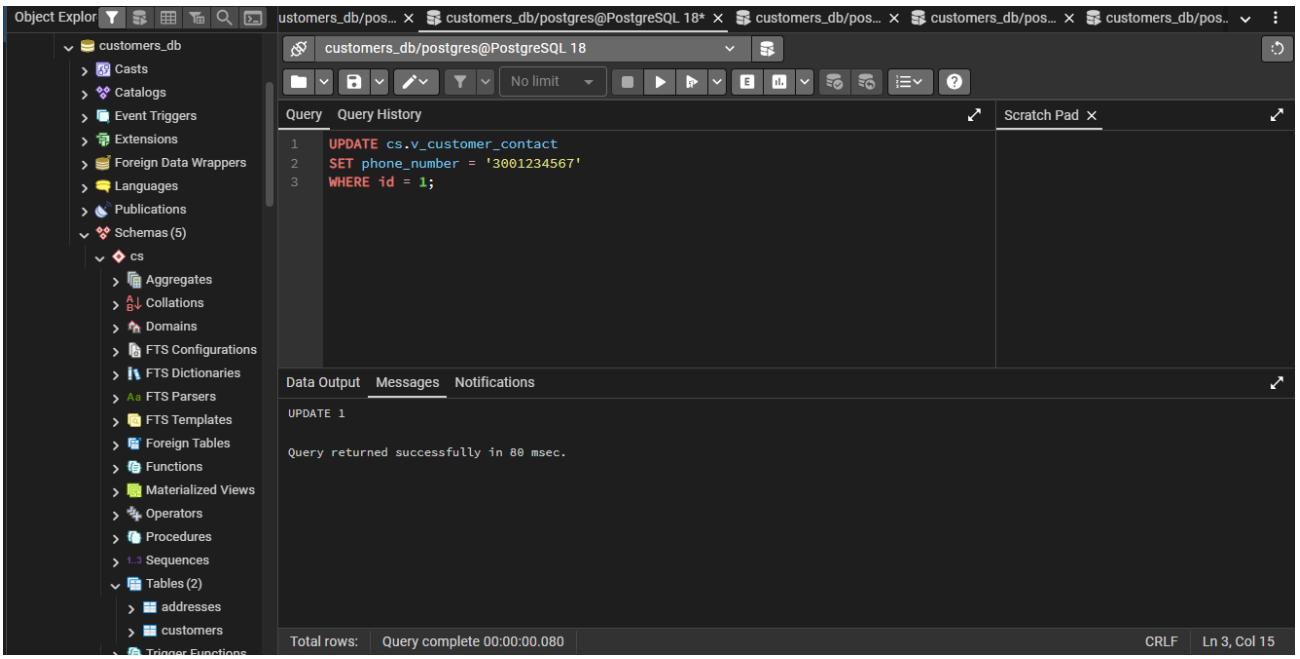
Image 10



b) Realizar un UPDATE a través de la vista para modificar el número de teléfono

```
UPDATE cs.v_customer_contact
SET phone_number = '3001234567'
WHERE id = 1;
```

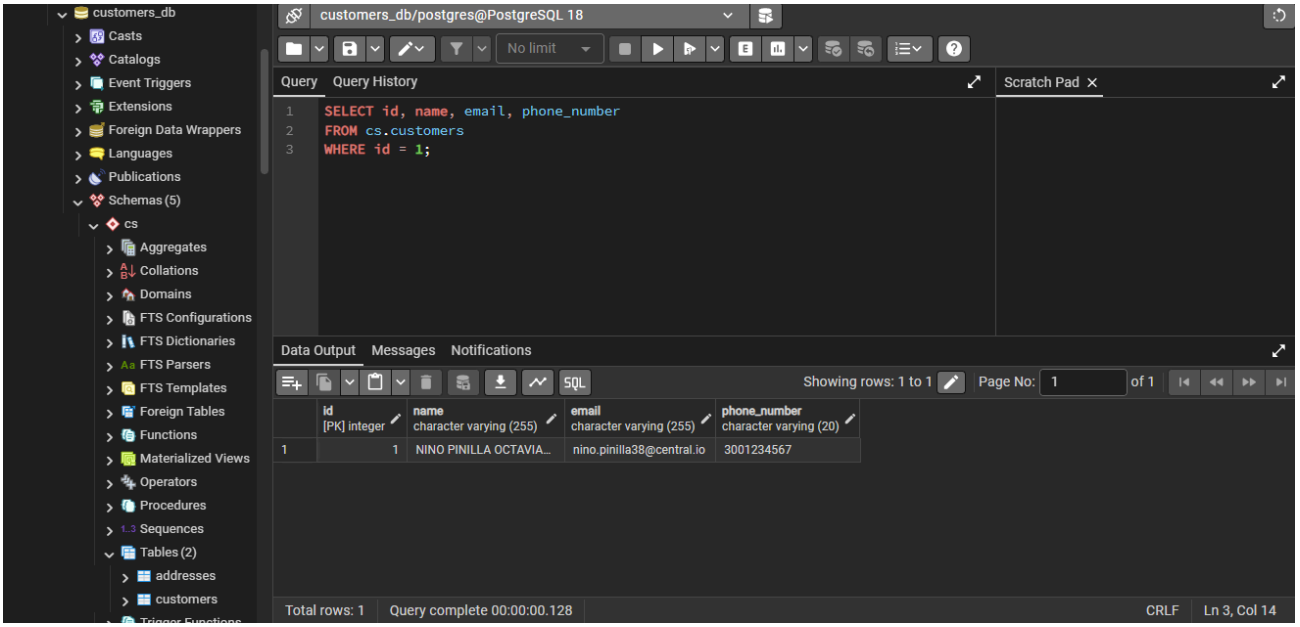
Image 11



c) Verificar mediante un SELECT directo sobre `cs.customers`

```
SELECT id, name, email, phone_number
FROM cs.customers
WHERE id = 1;
```

Image 12



d) Explicación de `WITH CHECK OPTION`

WITH CHECK OPTION en una vista actualizable impide realizar INSERT o UPDATE que hagan que una fila deje de cumplir la condición , garantizando así que todo lo que se modifique a través de la vista siga siendo visible en ella.

Ejercicio 5. Vista materializada — Ventas consolidadas por producto

a) Crear la vista materializada `cs.mv_sales_by_product`

```
CREATE MATERIALIZED VIEW cs.mv_sales_by_product AS
SELECT
    T2.name AS product_name,
    SUM(T1.quantity) AS total_units_sold,
    SUM(T1.quantity * T2.usd_price) AS total_revenue_usd,
    SUM(T1.quantity * T2.cop_price) AS total_revenue_cop
FROM pay.order_items T1
INNER JOIN ctg.products T2 ON T1.product_id = T2.id
GROUP BY T2.id, T2.name;
```

Image 13

customers_db/postgres@PostgreSQL 18

```
CREATE MATERIALIZED VIEW cs.mv_sales_by_product AS
SELECT
    T2.name AS product_name,
    SUM(T1.quantity) AS total_units_sold,
    SUM(T1.quantity * T2.usd_price) AS total_revenue_usd,
    SUM(T1.quantity * T2.cop_price) AS total_revenue_cop
FROM pay.order_items T1
INNER JOIN ctg.products T2 ON T1.product_id = T2.id
GROUP BY T2.id, T2.name;
```

```
SELECT 75

Query returned successfully in 119 msec.
```

Total rows: Query complete 00:00:00.119 CRLF Ln 9, Col 25

b) Consultar la vista y registrar el resultado inicial

```
SELECT * FROM cs.mv_sales_by_product;
```

Image 14

customers_db/postgres@PostgreSQL 18

```
SELECT * FROM cs.mv_sales_by_product;
```

Showing rows: 1 to 75 Page No: 1 of 1

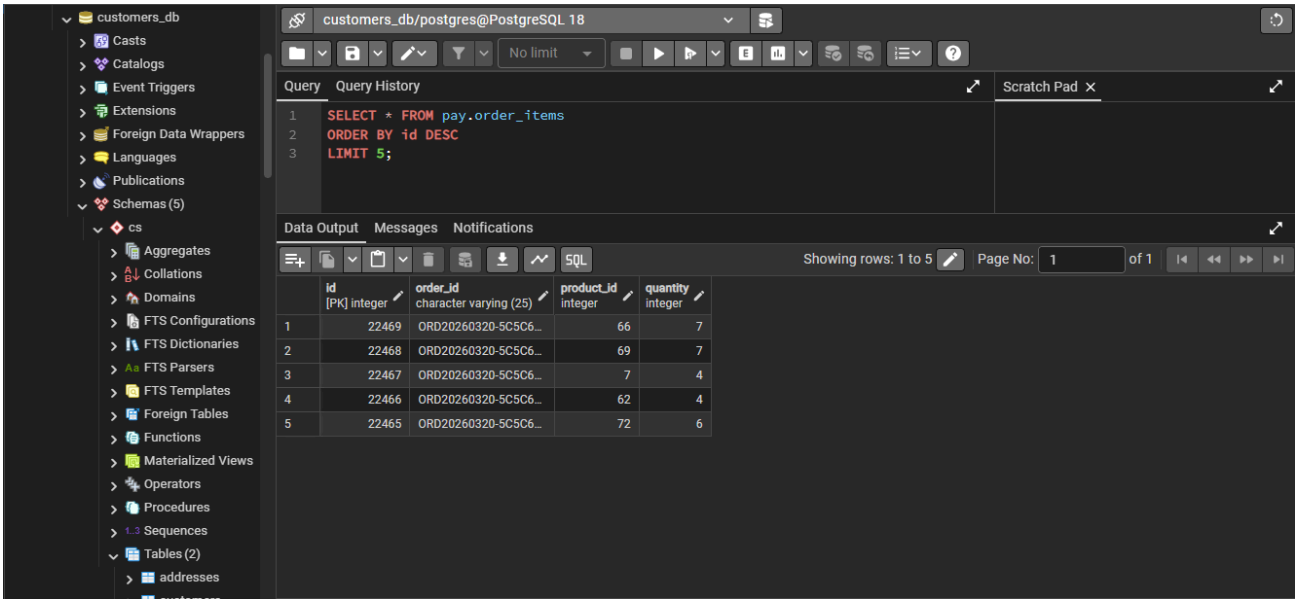
	product_name character varying (255)	total_units_sold bigint	total_revenue_usd numeric	total_revenue_cop numeric
1	Soporte Magnético para Auto Belkin	1487	37160.13	139350487.50
2	Funda Silicona iPhone 14	1425	18510.75	69415312.50
3	Apple Watch SE 2da Gen	1433	356817.00	1338063750.00
4	Aspiradora Robot Roomba i3	1501	450284.99	1688568712.50
5	Antivirus Kaspersky Premium 1 año	1300	38987.00	146201250.00
6	Convertidor USB-C a HDMI	1377	30294.00	113602500.00
7	Impresora HP DeskJet 2775	1560	138840.00	520650000.00
8	Mousepad Gamer RGB Corsair	1194	53730.00	201487500.00
9	Smartwatch Samsung Galaxy Watc...	1403	350735.97	1315259887.50
10	Aire Purificador Xiaomi Mi Air 3H	1264	113747.36	426552600.00
11	Licencia Microsoft Office 365 Perso...	1414	98965.86	371121975.00
12	Foco Inteligente TP-Link Tapo L530	1305	20866.95	78251062.50

Total rows: 75 Query complete 00:00:00.118 CRLF Ln 1, Col 38

c) Insertar un nuevo `order_item` y verificar persistencia

Si insertamos un nuevo registro en `pay.order_items`, el nuevo dato **no** se reflejará inmediatamente en la vista materializada. Esto se debe a que las vistas materializadas almacenan físicamente los datos en disco en el momento de su creación (o último refresco), actuando como un "snapshot" estático que no se vincula en tiempo real con la tabla base.

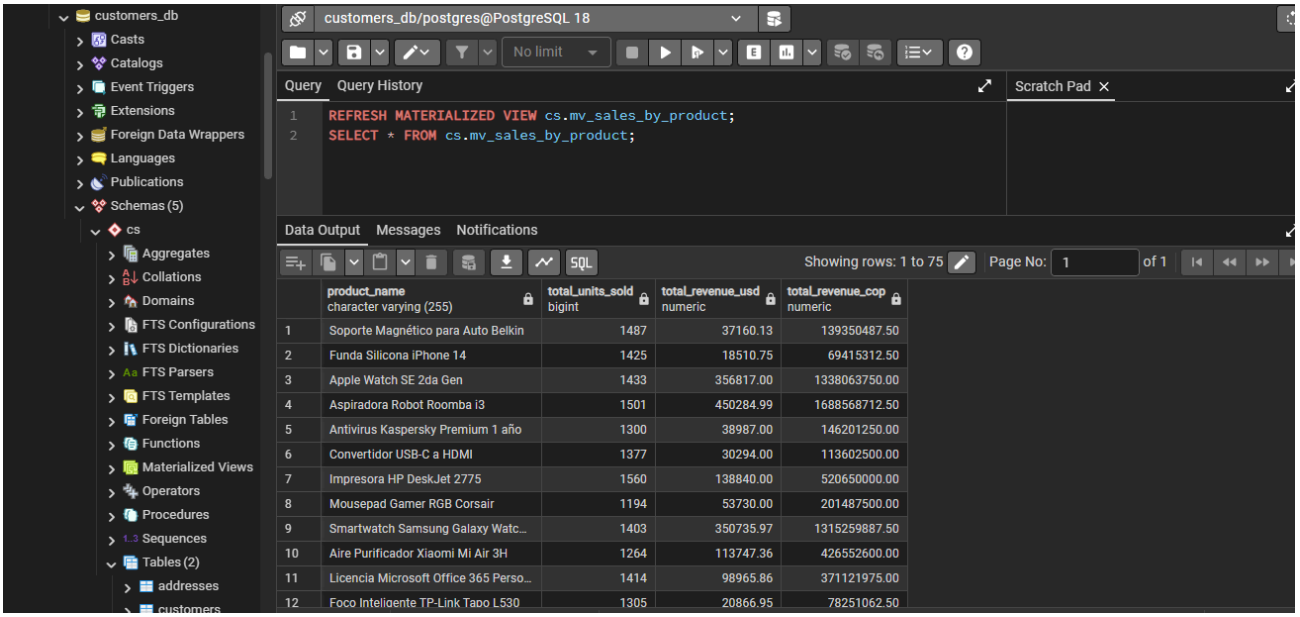
Image 15



d) Ejecutar REFRESH y consultar nuevamente

```
REFRESH MATERIALIZED VIEW cs.mv_sales_by_product;
SELECT * FROM cs.mv_sales_by_product;
```

Image 16



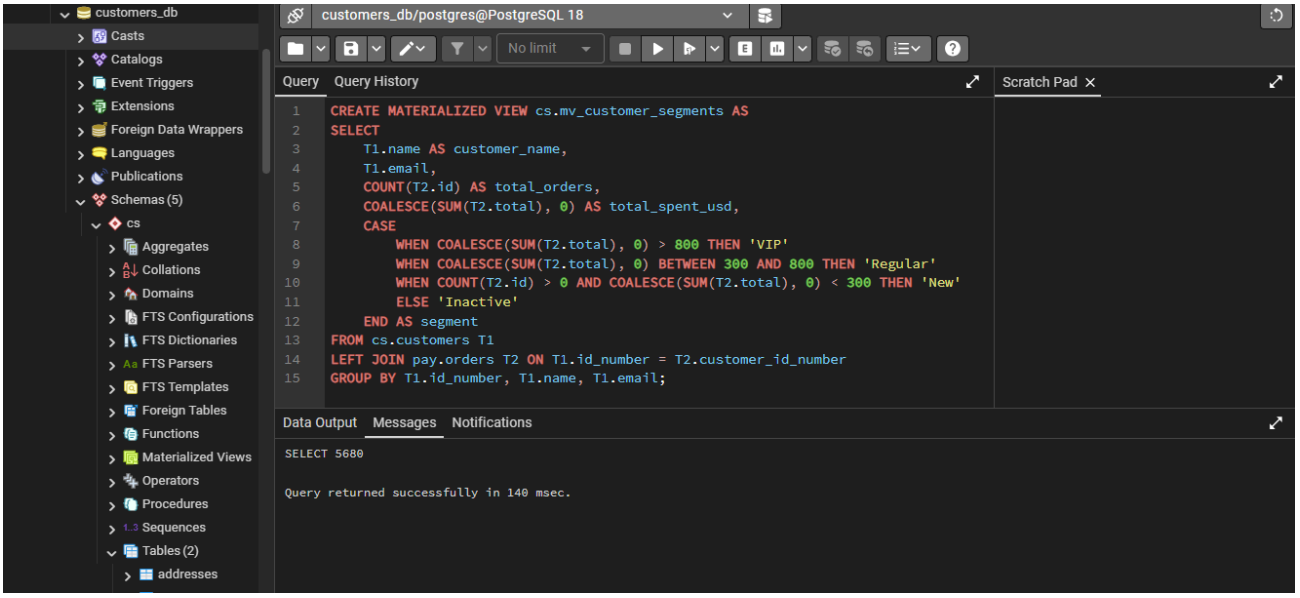
Ejercicio 6. Vista materializada — Segmentación de clientes

a) Crear la vista materializada `cs.mv_customer_segments`

```
CREATE MATERIALIZED VIEW cs.mv_customer_segments AS
SELECT
    T1.name AS customer_name,
    T1.email,
```

```sql
    COUNT(T2.id) AS total_orders,
    COALESCE(SUM(T2.total), 0) AS total_spent_usd,
    CASE
        WHEN COALESCE(SUM(T2.total), 0) > 800 THEN 'VIP'
        WHEN COALESCE(SUM(T2.total), 0) BETWEEN 300 AND 800 THEN 'Regular'
        WHEN COUNT(T2.id) > 0 AND COALESCE(SUM(T2.total), 0) < 300 THEN 'New'
        ELSE 'Inactive'
    END AS segment
FROM cs.customers T1
LEFT JOIN pay.orders T2 ON T1.id_number = T2.customer_id_number
GROUP BY T1.id_number, T1.name, T1.email;
```

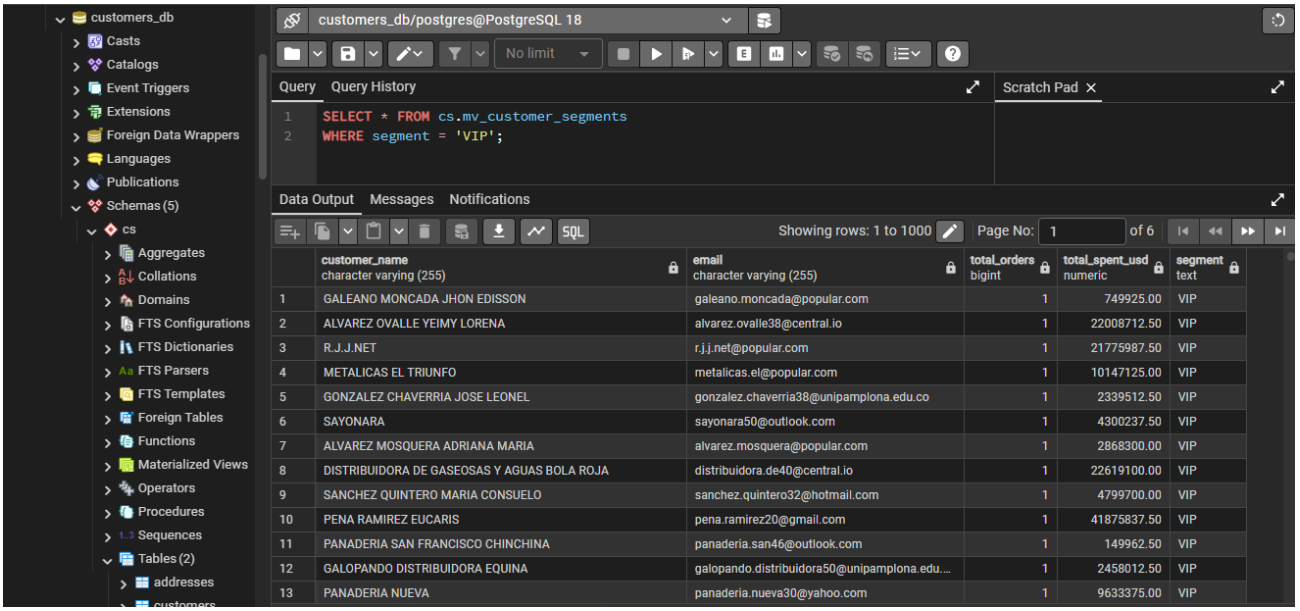
Image 17



b) Consultar únicamente los clientes del segmento VIP

```sql
SELECT * FROM cs.mv_customer_segments
WHERE segment = 'VIP';
```

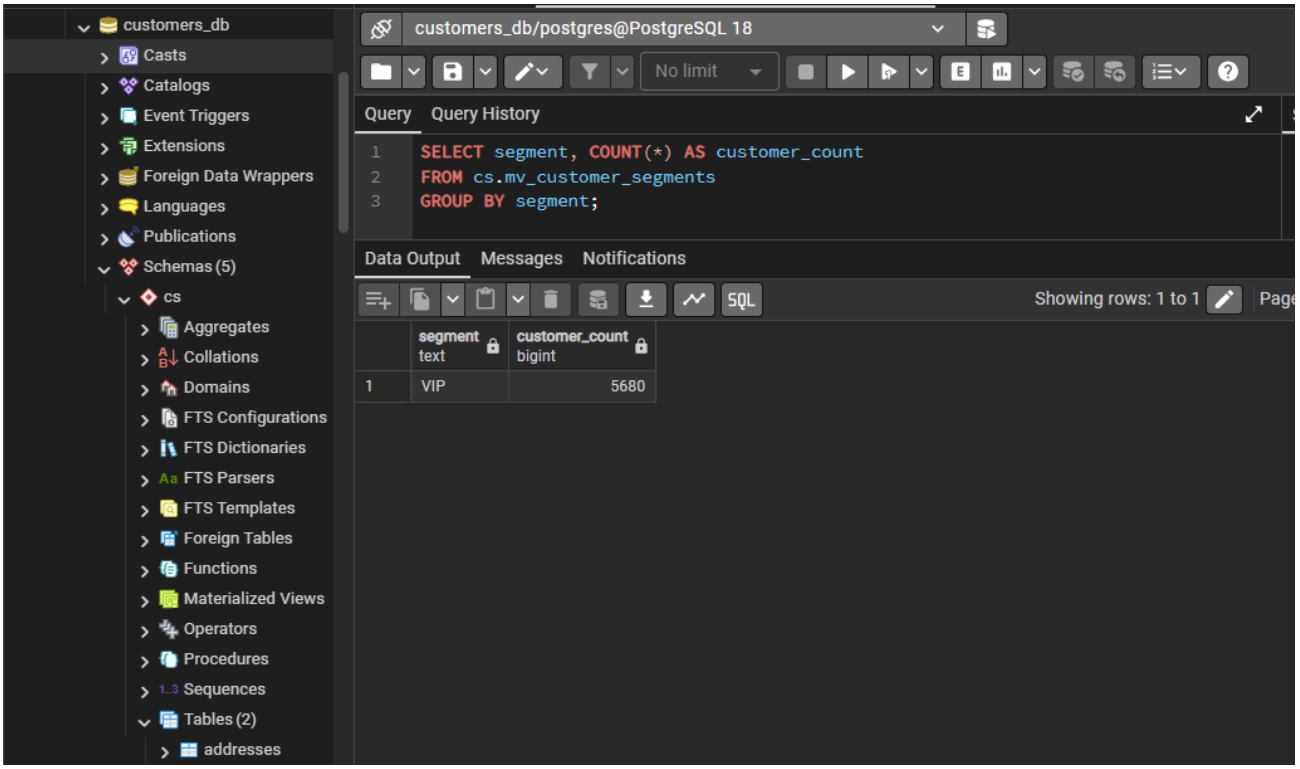
Image 18



c) Consultar cuántos clientes hay en cada segmento

```
SELECT segment, COUNT(*) AS customer_count
FROM cs.mv_customer_segments
GROUP BY segment;
```

Image 19



Ejercicio 7. Diseño — Vista recursiva

a) Descripción del caso de uso

En un entorno de comercio electrónico, los productos suelen pertenecer a categorías que están organizadas jerárquicamente (árbol de categorías). Por ejemplo: "Electrónica" > "Computación" > "Laptops". Una vista recursiva permite consultar toda la estructura de categorías, desde la raíz hasta el nivel más profundo, facilitando la navegación por el catálogo.

b) Definición de la nueva tabla

```sql
CREATE TABLE ctg.categories_hierarchy (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    parent_id INT,
    FOREIGN KEY (parent_id) REFERENCES ctg.categories_hierarchy(id)
);
```

c) Datos de ejemplo

id	name	parent_id
1	Electronics	NULL
2	Computing	1
3	Audio	1
4	Laptops	2
5	PC Components	2
6	Headphones	3

d) Nombre propuesto para la vista recursiva

`cs.rv_categories_path`

e) Descripción de los campos

- `category_id` : Identificador de la categoría.
- `category_name` : Nombre de la categoría actual.
- `level` : Profundidad en el árbol (1 para raíz, 2 para hijos, etc.).
- `path` : La ruta completa desde la raíz hasta la categoría (ej: "Electronics / Computing / Laptops").

f) Justificación

Este caso no puede resolverse con una vista normal porque SQL estándar es iterativo, no permite autoreferencias sin saber de antemano el número de niveles de profundidad. Una vista recursiva utiliza una CTE recursiva para "recorrer" el árbol de forma dinámica hasta que no existan más hijos, permitiendo escalar a cualquier número de niveles de subcategorías sin cambiar la consulta.